Room XI Connect - Complete Production Launch Roadmap

From 82% to 100% Production Ready

Overview

This document provides a complete, step-by-step action plan to take Room XI Connect from 82% to 100% production ready. The work is organized into four phases spanning 2-3 weeks, with clear deliverables and success criteria for each phase.

Phase 1: Infrastructure Setup (Days 1-3)

1.1 Database Setup

Task: Provision and configure Neon PostgreSQL production database

Steps:

1. Create Neon account at https://console.neon.tech
2. Create new project for production
3. Note the connection string (format: `postgresql://user:password@host/dbname`)
4. Create database backups configuration
5. Enable automatic backups (daily)
6. Set up connection pooling (recommended: 10-20 connections)

Verification:

```bash
# Test connection
psql "your-neon-connection-string" -c "SELECT version();"
```

Success Criteria: ✅ Can connect to production database

1.2 Email Service Setup

Task: Configure email provider for guardian verification and notifications

Option A: Gmail SMTP (Free, Recommended for MVP)

1. Create Gmail account or use existing one
2. Enable 2-Factor Authentication on Gmail account
3. Generate App Password:
   - Go to https://myaccount.google.com/apppasswords
   - Select "Mail" and "Windows Computer"
   - Copy the 16-character app password
4. Add to `.env.production`:

```
Plain Text

EMAIL_PROVIDER=gmail
GMAIL_USER=your-email@gmail.com
GMAIL_APP_PASSWORD=your-16-char-password
EMAIL_FROM=noreply@roomxiconnect.org
EMAIL_FROM_NAME=Room XI Connect
```

Option B: SendGrid (Paid, ~$15/month, Recommended for Scale)

1. Create SendGrid account at https://sendgrid.com
2. Create API key with Mail Send permission
3. Add to `.env.production`:

```
Plain Text

EMAIL_PROVIDER=sendgrid
SENDGRID_API_KEY=SG.your_api_key_here
EMAIL_FROM=noreply@roomxiconnect.org
EMAIL_FROM_NAME=Room XI Connect
```

Testing:

```
Bash

# Test email sending
curl -X POST http://localhost:3000/api/test-email \
  -H "Content-Type: application/json" \
  -d '{"to":"your-email@example.com"}'
```

Success Criteria: ✅ Test email received successfully

1.3 Redis Setup (Rate Limiting)

Task: Set up Upstash Redis for rate limiting

1. Create account at https://console.upstash.com
2. Create new Redis database
3. Copy REST URL and REST Token
4. Add to `.env.production` :

```
Plain Text

UPSTASH_REDIS_REST_URL=https://your-db.upstash.io
UPSTASH_REDIS_REST_TOKEN=your_token_here
```

Testing:

```
Bash

# Test Redis connection
curl -H "Authorization: Bearer your_token" \
  https://your-db.upstash.io/ping
```

Success Criteria: ✅ PONG response received

1.4 AI/LLM Configuration

Task: Configure Replit AI (or alternative LLM provider)

For Replit AI:

1. Ensure you have Replit account access
2. Get API endpoint and key from Replit
3. Add to `.env.production` :

```
Plain Text

OPENAI_API_BASE=https://api.replit.com/v1
OPENAI_API_KEY=your_replit_api_key
LLM_MODEL=gpt-4o-mini
```

For OpenAI (Alternative):

1. Create account at https://openai.com
2. Create API key at https://platform.openai.com/api-keys
3. Add to `.env.production` :

Plain Text

```
OPENAI_API_BASE=https://api.openai.com/v1
OPENAI_API_KEY=sk-your_openai_key
LLM_MODEL=gpt-4-turbo
```

Testing:

Bash

```
# Test LLM connection
curl -X POST http://localhost:3000/api/ximi/test \
  -H "Content-Type: application/json" \
  -d '{"message":"Hello"}'
```

Success Criteria: ✅ Ximi responds to test message

1.5 Environment Configuration

Task: Create and validate production environment file

Create `.env.production`:

Bash

```
# Copy .env.example to .env.production
cp .env.example .env.production

# Edit with production values
nano .env.production
```

Required Variables Checklist:

- [ ] `NODE_ENV=production`
- [ ] `DATABASE_URL=postgresql://...` (Neon)
- [ ] `EMAIL_PROVIDER=gmail` or `sendgrid`
- [ ] `GMAIL_USER` and `GMAIL_APP_PASSWORD` (if Gmail)
- [ ] `SENDGRID_API_KEY` (if SendGrid)
- [ ] `UPSTASH_REDIS_REST_URL` and `UPSTASH_REDIS_REST_TOKEN`
- [ ] `OPENAI_API_KEY` and `OPENAI_API_BASE`
- [ ] `SESSION_SECRET` (generate: `openssl rand -base64 32`)

- [ ] `JWT_SECRET` (generate: `openssl rand -base64 32`)
- [ ] `VITE_API_URL=https://yourdomain.com` (production domain)
- [ ] `VITE_APP_NAME=Room XI Connect`
- [ ] `VITE_APP_DESCRIPTION=...`

Validation:

```bash
# Check all required variables are set
npm run validate:env:production
```

Success Criteria: ✅ All required environment variables configured

---

Phase 2: Testing & Verification (Days 4-7)

2.1 Run Full Test Suite

Task: Execute all E2E tests to ensure everything works

```bash
# Install dependencies
npm install

# Run all E2E tests
npm run test:e2e

# Run specific test suites
npm run test:e2e:api          # API-only tests (124 tests)
npm run test:e2e:routes       # Route accessibility tests
npm run smoke:privacy         # Privacy compliance tests
npm run smoke:consent         # Consent enforcement tests
```

Expected Results:

- ✅ All 124 API tests pass
- ✅ All route tests pass
- ✅ Privacy smoke tests pass
- ✅ Consent enforcement tests pass

If Tests Fail:

1. Review test output for specific failures
2. Check environment variables
3. Verify database connectivity
4. Check email configuration
5. Review error logs

Success Criteria: ✅ All tests pass

2.2 Manual Testing Checklist

Youth Portal:

- [ ] Signup flow works (with guardian email if under 18)
- [ ] Login works
- [ ] Mood check-in works
- [ ] Mood history displays correctly
- [ ] Program discovery works
- [ ] Program map displays
- [ ] QR code scanner works
- [ ] Ximi chat works
- [ ] Crisis detection works
- [ ] Offline mode works (toggle airplane mode)
- [ ] Settings page works
- [ ] Logout works

Guardian Portal:

- [ ] Guardian receives verification email
- [ ] Guardian can verify via email link
- [ ] Guardian can view youth data (with consent)
- [ ] Guardian can manage consent preferences
- [ ] Guardian can request data export

Admin Portal:

- [ ] Login as admin works

- [ ] Dashboard loads with correct stats
- [ ] User list displays
- [ ] Audit logs show recent actions
- [ ] Program management works
- [ ] Analytics charts render
- [ ] Data export works

Organization Portal:

- [ ] Organization login works
- [ ] Can create/edit programs
- [ ] Can view attendance
- [ ] Can generate reports

Success Criteria: ✅ All manual tests pass

2.3 Performance Testing

Task: Verify application performance under load

Local Load Test:

```
Bash

# Run load test script
npm run test:load
```

Expected Results:

- ✅ API response time < 200ms
- ✅ Database queries < 100ms
- ✅ Can handle 100+ concurrent users
- ✅ No memory leaks detected

Browser Performance:

- [ ] Lighthouse score > 80
- [ ] First Contentful Paint < 2s
- [ ] Largest Contentful Paint < 3s

- [ ] Cumulative Layout Shift < 0.1

Success Criteria: ✅ Performance meets targets

2.4 Security Audit

Task: Conduct security review before launch

Checklist:

- [ ] Run `npm audit` and fix vulnerabilities
- [ ] Review SECURITY_ARCHITECTURE.md
- [ ] Test account lockout (5 failed attempts)
- [ ] Test session timeout (30 min inactivity)
- [ ] Test password strength validation
- [ ] Test CSRF protection
- [ ] Test SQL injection prevention
- [ ] Test XSS prevention
- [ ] Verify HTTPS/SSL configuration
- [ ] Check secure cookie flags
- [ ] Verify rate limiting works
- [ ] Test data encryption
- [ ] Review audit logs

Fix Vulnerabilities:

```
Bash

# Check vulnerabilities
npm audit

# Fix automatically
npm audit fix

# Fix with breaking changes if needed
npm audit fix --force
```

Success Criteria: ✅ No critical vulnerabilities, security audit passed

---

2.5 Accessibility Audit

Task: Verify WCAG 2.1 AA compliance

Automated Testing:

```
Bash

# Run accessibility tests
npm run test:a11y
```

Manual Testing:

- [ ] Test with keyboard only (no mouse)
- [ ] Test with screen reader (NVDA or JAWS)
- [ ] Test high contrast mode
- [ ] Test reduced motion mode
- [ ] Verify color contrast ratios
- [ ] Check focus indicators
- [ ] Test form accessibility
- [ ] Verify heading hierarchy

Tools:

- axe DevTools (Chrome extension)
- WAVE (Web Accessibility Evaluation Tool)
- Lighthouse (built into Chrome DevTools)

Success Criteria: ✅ WCAG 2.1 AA compliant

Phase 3: Deployment Preparation (Days 8-10)

3.1 Build Verification

Task: Verify production build works correctly

```
Bash

# Create production build
npm run build

# Verify build output
```

```
ls -lh dist/

# Preview production build locally
npm run preview
```

Expected Output:

- ✅ Build completes without errors
- ✅ dist/ folder contains all assets
- ✅ Service worker generated
- ✅ PWA manifest generated
- ✅ No console errors in preview

Success Criteria: ✅ Production build verified

3.2 Database Migration

Task: Set up production database schema

Bash

```
# Push schema to production database
DATABASE_URL="your-production-url" npm run db:push

# Verify schema created
DATABASE_URL="your-production-url" npm run db:studio
```

Verification:

- [ ] All tables created
- [ ] All indexes created
- [ ] Session table exists
- [ ] Can query tables

Success Criteria: ✅ Database schema deployed

3.3 Deployment Platform Setup

Choose Deployment Platform:

Option A: Vercel (Recommended)

1. Create Vercel account at https://vercel.com

2. Import GitHub repository
3. Configure environment variables in Vercel dashboard
4. Set up automatic deployments from main branch
5. Configure domain

Option B: Netlify

1. Create Netlify account at https://netlify.com
2. Connect GitHub repository
3. Configure build settings:

```
Plain Text

Build command: npm run build
Publish directory: dist
```

4. Set environment variables
5. Configure domain

Option C: Self-Hosted (AWS, DigitalOcean, etc.)

1. Provision server (Ubuntu 22.04 recommended)
2. Install Node.js 18+
3. Install PostgreSQL client
4. Set up environment variables
5. Configure reverse proxy (Nginx)
6. Set up SSL certificate (Let's Encrypt)

Success Criteria: ✅ Deployment platform configured

3.4 Domain & SSL Configuration

Task: Set up production domain with SSL

Steps:

1. Purchase domain (if not already done)
2. Point domain to deployment platform
3. Set up SSL certificate (automatic with most platforms)
4. Test HTTPS access

5. Set up redirect from HTTP to HTTPS

Verification:

```
Bash

# Test HTTPS
curl -I https://yourdomain.com

# Should return 200 OK with HTTPS
```

Success Criteria: ✅ Domain accessible via HTTPS

3.5 Monitoring & Logging Setup

Task: Configure production monitoring

Error Tracking (Optional but Recommended):

1. Create Sentry account at https://sentry.io
2. Create project for Room XI Connect
3. Get DSN (Data Source Name)
4. Add to `.env.production` :

```
Plain Text

VITE_SENTRY_DSN=your_sentry_dsn
```

Application Logging:

- ✅ Pino logger configured
- ✅ Server logs to stdout
- ✅ Error logs captured
- ✅ Session logs recorded

Database Monitoring:

- ✅ Neon dashboard accessible
- ✅ Query performance visible
- ✅ Backup status verified

Success Criteria: ✅ Monitoring configured

Phase 4: Launch & Post-Launch (Days 11-14)

4.1 Final Pre-Launch Checklist

24 Hours Before Launch:

- [ ] All tests passing
- [ ] Security audit completed
- [ ] Performance verified
- [ ] Database backups configured
- [ ] Monitoring active
- [ ] Error tracking enabled
- [ ] Support contact available
- [ ] Incident response plan ready
- [ ] Rollback plan documented
- [ ] User communication prepared

Deployment Checklist:

- [ ] Production build created
- [ ] Environment variables set
- [ ] Database migrated
- [ ] SSL certificate valid
- [ ] Domain pointing correctly
- [ ] Monitoring active
- [ ] Error tracking enabled
- [ ] Logs aggregated

Success Criteria: ✅ All pre-launch checks passed

4.2 Launch Procedure

Launch Day:

1. **Backup Current Data:**

```
Bash
```

```
# Create database backup
pg_dump "your-production-url" > backup-$(date +%Y%m%d).sql
```

2. **Deploy to Production:**

```
Bash

# If using Vercel/Netlify: Push to main branch
git push origin main

# If self-hosted:
npm run build
npm start
```

3. **Verify Deployment:**
   - [ ] Website loads
   - [ ] No console errors
   - [ ] API endpoints respond
   - [ ] Database connected
   - [ ] Email working
   - [ ] Ximi responding
   - [ ] Admin dashboard accessible
4. **Monitor Closely:**
   - Watch error tracking for issues
   - Monitor performance metrics
   - Check user feedback
   - Be ready to rollback if needed

Success Criteria: ✅ Application live and functioning

4.3 Post-Launch Monitoring (First Week)

Daily Tasks:

- [ ] Check error logs for issues
- [ ] Monitor performance metrics
- [ ] Review user feedback

- [ ] Check database performance
- [ ] Verify backups running
- [ ] Monitor uptime

Weekly Tasks:

- [ ] Review analytics
- [ ] Check security logs
- [ ] Analyze user behavior
- [ ] Plan improvements
- [ ] Document issues

Success Criteria: ✅ No critical issues, system stable

4.4 Rollback Plan (If Needed)

If Critical Issues Occur:

1. **Immediate Actions:**

```bash
# Revert to previous version
git revert HEAD
git push origin main

# Or restore from backup
psql "your-production-url" < backup-YYYYMMDD.sql
```

2. **Communication:**
   - Notify users of issue
   - Provide status updates
   - Explain resolution timeline
3. **Post-Mortem:**
   - Document what went wrong
   - Identify root cause
   - Plan prevention

Success Criteria: ✅ System restored, users informed

Detailed Implementation Tasks

Task 1: Set Up Neon PostgreSQL

Time: 30 minutes

Steps:

```bash
# 1. Create Neon account
# Go to https://console.neon.tech and sign up

# 2. Create new project
# Name: "Room XI Connect Production"
# Region: US East (or closest to your users )

# 3. Get connection string
# Copy from Neon dashboard

# 4. Test connection
psql "postgresql://user:password@host/dbname" -c "SELECT version();"

# 5. Create .env.production with DATABASE_URL
echo "DATABASE_URL=postgresql://user:password@host/dbname" >> .env.production

# 6. Push schema
DATABASE_URL="your-connection-string" npm run db:push
```

Verification:

- ✅ Can connect to database
- ✅ Schema tables created
- ✅ Can query tables

Task 2: Configure Email Service

Time: 20 minutes (Gmail) or 30 minutes (SendGrid)

Gmail Setup:

```bash
# 1. Create/use Gmail account
# 2. Enable 2FA at https://myaccount.google.com/security
# 3. Generate app password at https://myaccount.google.com/apppasswords
```

```
# 4. Add to .env.production:

cat >> .env.production << EOF
EMAIL_PROVIDER=gmail
GMAIL_USER=your-email@gmail.com
GMAIL_APP_PASSWORD=your-16-char-password
EMAIL_FROM=noreply@roomxiconnect.org
EMAIL_FROM_NAME=Room XI Connect
EOF

# 5. Test email
curl -X POST http://localhost:3000/api/test-email \
  -H "Content-Type: application/json" \
  -d '{"to":"test@example.com"}'
```

Verification:

- ✅ Test email received
- ✅ Email formatting correct
- ✅ Links work in email

Task 3: Set Up Redis for Rate Limiting

Time: 15 minutes

Steps:

Bash

```
# 1. Create Upstash account at https://console.upstash.com
# 2. Create Redis database
# 3. Copy REST URL and Token
# 4. Add to .env.production:

cat >> .env.production << EOF
UPSTASH_REDIS_REST_URL=https://your-db.upstash.io
UPSTASH_REDIS_REST_TOKEN=your_token_here
EOF

# 5. Test connection
curl -H "Authorization: Bearer your_token" \
  https://your-db.upstash.io/ping
```

Verification:

- ✅ PONG response received

- ✅ Can set/get keys
- ✅ Rate limiting works

Task 4: Configure LLM Provider

Time: 10 minutes

Steps:

```bash
# 1. Get API key from Replit or OpenAI
# 2. Add to .env.production:

cat >> .env.production << EOF
OPENAI_API_BASE=https://api.replit.com/v1
OPENAI_API_KEY=your_api_key
LLM_MODEL=gpt-4o-mini
EOF

# 3. Test connection
curl -X POST http://localhost:3000/api/ximi/test \
  -H "Content-Type: application/json" \
  -d '{"message":"Hello"}'
```

Verification:

- ✅ Ximi responds to messages
- ✅ Crisis detection works
- ✅ No API errors

Task 5: Run Full Test Suite

Time: 30 minutes

Steps:

```bash
# 1. Install dependencies
npm install

# 2. Run all tests
npm run test:e2e
```

```
# 3. Check results
# Should see: "X passed" for all tests

# 4. If tests fail, debug:
npm run test:e2e:api  # API tests only
npm run test:e2e:routes  # Route tests only
npm run smoke:privacy  # Privacy tests only
```

Verification:

- ✅ All 124+ tests pass
- ✅ No test failures
- ✅ No console errors

Task 6: Security Audit

Time: 1-2 hours

Steps:

Bash

```
# 1. Check npm vulnerabilities
npm audit

# 2. Fix vulnerabilities
npm audit fix

# 3. Review security architecture
cat SECURITY_ARCHITECTURE.md

# 4. Test security features:
# - Account lockout (5 failed attempts )
# - Session timeout (30 min inactivity)
# - Password strength validation
# - CSRF protection
# - SQL injection prevention
# - XSS prevention

# 5. Manual security testing
# - Try weak password (should fail)
# - Try SQL injection in login (should fail)
# - Try XSS in forms (should fail)
# - Try CSRF attack (should fail)
```

Verification:

- ✅ No critical vulnerabilities
- ✅ All security tests pass
- ✅ No security issues found

Task 7: Deploy to Production

Time: 30 minutes

Steps (Vercel):

```
Bash

# 1. Push code to GitHub
git add .
git commit -m "Production deployment"
git push origin main

# 2. Vercel auto-deploys
# Monitor at https://vercel.com/dashboard

# 3. Verify deployment
curl https://yourdomain.com
```

Steps (Self-Hosted):

```
Bash

# 1. Create production build
npm run build

# 2. Start production server
NODE_ENV=production npm start

# 3. Verify server running
curl http://localhost:3000
```

Verification:

- ✅ Website loads
- ✅ API responds
- ✅ Database connected
- ✅ No errors in logs

Task 8: Set Up Monitoring

Time: 30 minutes

Steps:

```bash
# 1. Create Sentry account (optional )
# Go to https://sentry.io

# 2. Create project for Room XI Connect

# 3. Get DSN and add to .env.production:
echo "VITE_SENTRY_DSN=your_dsn" >> .env.production

# 4. Verify monitoring working:
# - Check Sentry dashboard
# - Trigger test error
# - Verify error appears in Sentry

# 5. Set up log aggregation:
# - Configure Neon logs
# - Set up application logs
# - Create dashboards
```

Verification:

- ✅ Errors captured in Sentry
- ✅ Logs aggregated
- ✅ Dashboards created

Success Criteria for 100% Production Ready

Infrastructure ✅

- [ ] Neon PostgreSQL database provisioned
- [ ] Email service configured (Gmail or SendGrid)
- [ ] Redis (Upstash) configured for rate limiting
- [ ] LLM provider configured (Replit AI or OpenAI)
- [ ] All environment variables set
- [ ] SSL certificate installed

- [ ] Domain configured
- [ ] Backups configured

Testing ✅

- [ ] All 124+ E2E tests passing
- [ ] Manual testing completed
- [ ] Performance testing passed
- [ ] Security audit passed
- [ ] Accessibility audit passed
- [ ] Load testing passed

Deployment ✅

- [ ] Production build created
- [ ] Database schema deployed
- [ ] Monitoring configured
- [ ] Error tracking enabled
- [ ] Logging configured
- [ ] Rollback plan ready

Launch ✅

- [ ] All pre-launch checks passed
- [ ] Deployment successful
- [ ] Website live and accessible
- [ ] All features working
- [ ] No critical errors
- [ ] Users can sign up and use app

Timeline Summary

Phase	Duration	Key Deliverables

Phase 1: Infrastructure	Days 1-3	Database, email, Redis, LLM, env vars
Phase 2: Testing	Days 4-7	All tests passing, security audit, performance verified
Phase 3: Deployment Prep	Days 8-10	Build verified, database migrated, platform configured
Phase 4: Launch	Days 11-14	Live deployment, monitoring active, stable operation

Total Time: 2-3 weeks to 100% production ready

Support & Troubleshooting

Common Issues

Issue: Email not sending

- Check GMAIL_APP_PASSWORD is correct
- Verify Gmail account has 2FA enabled
- Check email logs for errors
- Try SendGrid as alternative

Issue: Database connection fails

- Verify DATABASE_URL is correct
- Check Neon dashboard for connection limits
- Verify firewall allows connections
- Test with psql directly

Issue: Tests failing

- Check environment variables set
- Verify database connected
- Check email service working
- Review test output for specific errors

Issue: Performance slow

- Check database query performance

- Verify Redis working
- Check server resources
- Review application logs

Getting Help

- Review DEVELOPER_HANDOFF.md for detailed documentation
- Check SECURITY_ARCHITECTURE.md for security questions
- Review test files for examples
- Check server logs for errors

Final Checklist Before Launch

One Week Before:

- [ ] All infrastructure set up
- [ ] All tests passing
- [ ] Security audit completed
- [ ] Performance verified
- [ ] Team trained on deployment

One Day Before:

- [ ] Final test run
- [ ] Backups verified
- [ ] Monitoring tested
- [ ] Incident response plan reviewed
- [ ] Support contact available

Launch Day:

- [ ] Deploy to production
- [ ] Verify all features working
- [ ] Monitor for errors
- [ ] Be ready to rollback
- [ ] Communicate status to users

First Week:

- [ ] Monitor error logs daily
- [ ] Check performance metrics
- [ ] Review user feedback
- [ ] Fix any critical issues
- [ ] Plan improvements

Conclusion

Following this roadmap will take Room XI Connect from 82% to 100% production ready in 2-3 weeks. The key is to follow each phase systematically, verify each step, and don't skip testing.

You've built an excellent application. This roadmap will get it to production safely and successfully.

Good luck with the launch! 🚀